SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

B. TECH – COMPUTER SCIENCE AND ENGINEERING

PYTHON PROGRAMMING

MINI PROJECT REPORT

RETAIL SALES DASHBOARD USING CSV AND MATPLOTLIB

A Python-based system for retail transaction analysis, business metrics and data visualization

Name	HAIKRIVA R B
USN	25BTRPW043
Class / Section	CSE-PWC(A)
Subject Name	Python Programming
Faculty Name	Ms. Sujithra G
Date of Report Submission	02/11/2026
Project Topic	Retail Sales Dashboard Using CSV and Matplotlib
Project Components	CSV Processing, Matplotlib and Flask

School of Computer Science and Engineering

TABLE OF CONTENTS

S. No.	Topic	Page No.
1	Introduction	3
2	Problem Statement	3
3	Objectives	3
4	Project Description	4
5	Concepts Used in the Program	4–5
6	Algorithm	5
7	Flowchart	6
8	Python Program – Core Implementation	7–9
9	Output and Dashboard	10
10	Conclusion	11

Project overview: This project reads retail transaction data from a CSV file, processes the records using Python, calculates sales and profitability measures, and presents the results through Matplotlib charts and an interactive Flask dashboard.

1. INTRODUCTION

Retail businesses generate transaction data whenever customers purchase products. Each transaction may contain details such as date, product, category, quantity, unit price, discount, cost, payment method, city, order status and customer rating. Studying these records helps a business understand its sales performance and customer behaviour.

The **Retail Sales Dashboard Using CSV and Matplotlib** is a Python mini project that converts transaction records into useful business information. The project uses a CSV file as its data source. Python reads and processes the records, calculates key performance indicators (KPIs), groups sales information and creates visual charts using Matplotlib.

The project also provides a command-line menu and a browser-based dashboard developed with Flask. Users can view an executive summary, filter records by category or city, generate a Matplotlib dashboard, export a text report, regenerate sample data and explore the interactive dashboard.

The uploaded dataset contains 1,201 retail transactions dated during 2025 and covers six product categories and eight cities. The figures in this report are calculated from the supplied CSV file and the category configuration currently included in the project.

2. PROBLEM STATEMENT

Retail transaction data stored in CSV files can be difficult to interpret when it is viewed row by row. Manual calculations of revenue, profit, product performance and customer patterns take time and may lead to mistakes. A simple system is therefore required to read the CSV data, process it automatically, summarize important measures and display the results in charts.

The problem is to design and develop a Python-based Retail Sales Dashboard that loads retail transaction records from a CSV file, calculates sales and profitability indicators, supports category and city analysis, and presents the results through Matplotlib visualizations and an interactive web interface.

3. OBJECTIVES

- To read retail transaction records from a CSV file using Python.
- To calculate total revenue, net profit, profit margin, average order value and other KPIs.
- To analyse sales by month, product category, city, payment method and day of the week.
- To identify top-performing products and customer segments.
- To create a multi-chart sales dashboard using Matplotlib.
- To provide filtering, report export and dataset regeneration options.
- To provide an interactive browser dashboard using the Flask framework.

4. PROJECT DESCRIPTION

The project is organized into separate Python modules. This modular structure keeps data generation, data analysis, chart creation and user interaction in different files. The main program acts as the controller and connects the modules.

4.1 Data Source and Loading

The file **retail_sales_data.csv** stores the transaction records. The dataset includes transaction ID, date, time, customer ID, gender, age group, city, product category, product name, quantity, unit price, unit cost, discount percentage, total amount, cost total, net profit, payment method, order status and rating. The analyzer reads the file with **csv.DictReader**, which makes each row accessible through its column headings.

4.2 Data Processing and Analysis

The RetailSalesAnalyzer class loads the CSV records and prepares values such as dates, quantities and monetary amounts. It calculates KPIs and groups records using dictionaries and defaultdict. It can filter records by category, city, date range or quarter. Category configuration is stored in a JSON file and can adjust selling price, cost, discount, target revenue, category name and active status.

4.3 Dashboard Generation

The dashboard.py module uses Matplotlib to generate a large dashboard image. GridSpec arranges a KPI banner and six chart areas. The charts show monthly revenue and profit, category revenue and profit, payment method share, customer RFM segments, city performance and sales by day of the week. The generated image is saved as **retail_sales_dashboard.png**.

4.4 Interactive Web Application

The app.py module uses Flask to serve the browser interface. It provides routes for dashboard data, category configuration, simulated sales, report export, CSV export and CSV upload. The HTML interface is stored in templates/index.html. The browser dashboard uses the API data to update its visualizations and transaction table.

4.5 Main Project Files

File	Purpose
main.py	Command-line menu and application control
sales_analyzer.py	CSV loading, KPI calculations, filtering and analysis
dashboard.py	Matplotlib dashboard creation
generate_data.py	Generation of sample retail transaction records
app.py	Flask web application and API routes
category_config.json	Editable category settings
retail_sales_data.csv	Retail transaction dataset
templates/index.html	Browser dashboard interface

5. CONCEPTS USED IN THE PROGRAM

CSV File Handling: The csv module reads and writes comma-separated transaction data. csv.DictReader reads the header row and maps each record to a dictionary, which reduces the need to access fields by numeric column positions.

Functions: Functions divide the program into reusable tasks. Examples include create_dashboard(), generate_retail_sales_csv(), get_kpis(), get_monthly_sales() and generate_text_summary().

Classes and Objects: The RetailSalesAnalyzer class groups related data and analysis methods. An object of this class is created using the CSV file path and is used by the command-line and web application.

Lists and Dictionaries: Lists store collections of transaction records and chart labels. Dictionaries store each transaction, category configuration and grouped analysis results.

Loops and Conditional Statements: Loops process transaction rows and display menu options. if/elif/else statements select menu actions, validate choices and classify customer segments.

Exception Handling: try/except blocks handle invalid or incomplete transaction values while loading CSV records and protect selected file and web operations from terminating unexpectedly.

Matplotlib: Matplotlib creates line charts, bar charts, horizontal bar charts and a doughnut chart. These visualizations make changes and comparisons easier to understand.

GridSpec Layout: matplotlib.gridspec.GridSpec arranges the dashboard into sections with different sizes. This allows the KPI banner to use the full width while six charts are placed below it.

Flask and JSON: Flask provides web routes and returns analysis results as JSON. JSON is also used to save and load editable category settings.

Data Aggregation: collections.defaultdict is used to accumulate revenue, profit, quantities and counts by month, category, city, product and payment method. This supports efficient one-pass grouping.

5.1 Important Calculations

- **Total Revenue** = Sum of the transaction Total_Amount values.
- **Net Profit** = Total Revenue – Total Cost.
- **Profit Margin (%)** = (Net Profit ÷ Total Revenue) × 100.
- **Average Order Value** = Total Revenue ÷ Number of Transactions.
- **Month-over-Month Growth (%)** = ((Current Month Revenue – Previous Month Revenue) ÷ Previous Month Revenue) × 100.
- **Return Rate (%)** = (Number of Returned Transactions ÷ Total Transactions) × 100.

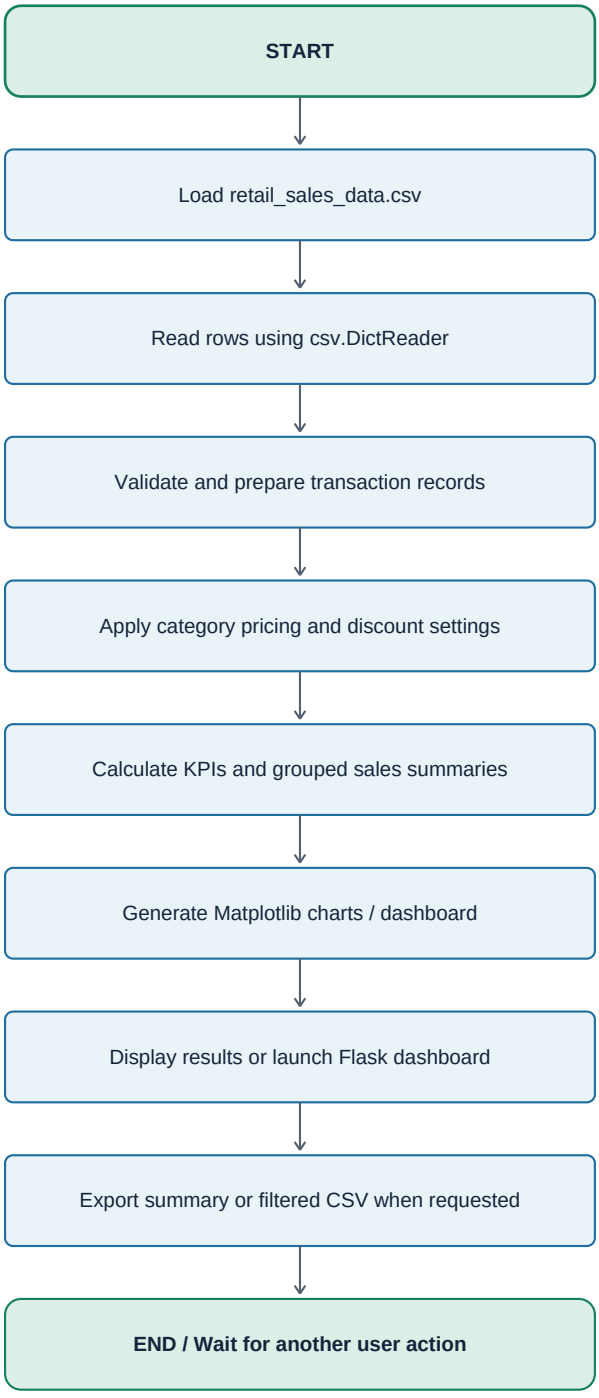
6. ALGORITHM

1. Start the application.
2. Check whether the retail CSV dataset exists. If it is missing, generate a sample dataset.
3. Create a RetailSalesAnalyzer object and load the CSV records.
4. Load the category configuration from the JSON file and apply the configured values.
5. Display the main menu and ask the user to select an operation.
6. If the user selects the executive summary, calculate and display the KPIs and analysis.
7. If the user selects the Matplotlib dashboard, collect the grouped data and draw the charts.
8. If the user selects filtering, apply the selected category and city and calculate filtered KPIs.
9. If the user selects report export, write the text summary to a report file.

10. If the user selects dataset regeneration, create new CSV records and reload the analyzer.
11. If the user selects the web dashboard, launch the Flask application.
12. Repeat the menu until the user selects Exit, then stop the program.

7. FLOWCHART

The following flowchart shows the main processing sequence of the Retail Sales Dashboard. The application loads the CSV data, prepares the records, performs analysis and then displays or exports the requested results.



Note: The command-line menu and Flask web interface provide different ways to access the same analysis engine. The flowchart represents the overall project workflow rather than every individual web route.

8. PYTHON PROGRAM – CORE IMPLEMENTATION (1/3)

The complete project is divided into multiple source files. The following selected sections show the central implementation logic used for CSV loading, data preparation and KPI calculations. The original source files are retained in the uploaded project ZIP.

8.1 Loading CSV Records

```
import csv
from datetime import datetime

def load_data(self):
    self.raw_records = []

    with open(self.csv_filepath, mode="r",
              encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            try:
                price = float(row.get("Price_Per_Unit", 0))
                qty = int(row.get("Quantity", 1))
                date_obj = datetime.strptime(
                    row["Date"], "%Y-%m-%d"
                )

                record = {
                    "Transaction_ID": row["Transaction_ID"],
                    "Date": date_obj,
                    "Customer_ID": row["Customer_ID"],
                    "City": row.get("City", "Unknown"),
                    "Product_Category": row["Product_Category"],
                    "Product_Name": row["Product_Name"],
                    "Quantity": qty,
                    "Price_Per_Unit": price,
                    "Total_Amount": float(
                        row.get("Total_Amount", price * qty)
                    )
                }
                self.raw_records.append(record)

            except (ValueError, KeyError):
                continue

    return len(self.raw_records)
```

The actual analyzer stores additional fields such as cost, discount, payment method, order status and customer rating. Invalid rows that cannot be converted or are missing required keys are skipped.

8.2 KPI Calculation

```
def get_kpis(self, records=None):
    data = records if records is not None else self.records

    total_revenue = sum(
        r["Total_Amount"] for r in data
    )
    total_profit = sum(
        r["Net_Profit"] for r in data
    )
    total_units = sum(r["Quantity"] for r in data)
    total_transactions = len(data)

    margin = (
        total_profit / total_revenue * 100
        if total_revenue > 0 else 0
    )
    avg_order_value = (
        total_revenue / total_transactions
        if total_transactions > 0 else 0
    )

    return {
        "total_revenue": round(total_revenue, 2),
        "total_profit": round(total_profit, 2),
        "profit_margin_pct": round(margin, 1),
        "total_units": total_units,
        "total_transactions": total_transactions,
        "avg_order_value": round(avg_order_value, 2)
    }
```

The full get_kpis() method also calculates discount percentage, return rate, average customer rating, unique customers, top category and top product.

8. PYTHON PROGRAM – CORE IMPLEMENTATION (2/3)

8.3 Grouping Sales by Category

```
from collections import defaultdict

def get_category_sales(self, records=None):
    data = records if records is not None else self.records
    cat_rev = defaultdict(float)
    cat_profit = defaultdict(float)
    cat_units = defaultdict(int)

    for row in data:
        category = row["Product_Category"]
        cat_rev[category] += row["Total_Amount"]
        cat_profit[category] += row["Net_Profit"]
        cat_units[category] += row["Quantity"]

    sorted_cats = sorted(
        cat_rev.items(),
        key=lambda item: item[1],
        reverse=True
    )

    categories = [item[0] for item in sorted_cats]
    revenues = [round(item[1], 2) for item in sorted_cats]
    profits = [round(cat_profit[c], 2) for c in categories]

    margins = [
        round(cat_profit[c] / cat_rev[c] * 100, 1)
        if cat_rev[c] > 0 else 0
        for c in categories
    ]

    return {
        "categories": categories,
        "revenues": revenues,
        "profits": profits,
        "units": [cat_units[c] for c in categories],
        "margins": margins
    }
```

8.4 Creating the Matplotlib Dashboard

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

def create_dashboard(csv_path, output_image,
                     show_plot=False):
    analyzer = RetailSalesAnalyzer(csv_path)
    kpis = analyzer.get_kpis()
    monthly = analyzer.get_monthly_sales()
    categories = analyzer.get_category_sales()

    fig = plt.figure(figsize=(19, 11.5))
    grid = gridspec.GridSpec(
        3, 3,
        height_ratios=[1.0, 3.2, 3.2],
        hspace=0.36, wspace=0.25
    )

    # Monthly revenue and profit trend
    ax1 = fig.add_subplot(grid[1, 0])
    x = range(len(monthly["months"]))
    ax1.plot(x, monthly["revenues"],
             marker="o", label="Revenue")
    ax1.plot(x, monthly["profits"],
             marker="s", linestyle="--",
             label="Net Profit")
    ax1.set_title("Monthly Revenue & Net Profit")
    ax1.legend()

    # Category revenue and profit comparison
    ax2 = fig.add_subplot(grid[1, 1])
    ax2.bar(categories["categories"],
            categories["revenues"], label="Revenue")
    ax2.set_title("Category Revenue")

    fig.savefig(output_image, dpi=300,
                bbox_inches="tight")
    if show_plot:
        plt.show()
    plt.close(fig)
```

The actual dashboard function adds the KPI banner, category profit comparison, payment method doughnut chart, RFM segment chart, city performance chart and day-of-week sales chart. It uses a GridSpec layout and saves the output at 300 DPI.

8. PYTHON PROGRAM – CORE IMPLEMENTATION (3/3)

8.5 Command-Line Menu

```
def main():
    csv_file = "retail_sales_data.csv"

    if not os.path.exists(csv_file):
        generate_retail_sales_csv(
            csv_file, num_records=1000
        )

    analyzer = RetailSalesAnalyzer(csv_file)

    while True:
        print_header()
        print_menu()
        choice = input("Enter your choice: ").strip()

        if choice == "1":
            print(analyzer.generate_text_summary())

        elif choice == "2":
            create_dashboard(
                csv_path=csv_file,
                output_image="retail_sales_dashboard.png",
                show_plot=True
            )

        elif choice == "3":
            filter_menu(analyzer)

        elif choice == "4":
            export_report_to_file(analyzer)

        elif choice == "5":
            # Regenerate dataset and reload analyzer
            generate_retail_sales_csv(csv_file)
            analyzer = RetailSalesAnalyzer(csv_file)

        elif choice == "6":
            from app import app
            app.run(debug=False, host="127.0.0.1",
                    port=5000)

        elif choice == "0":
            break
        else:
            print("Invalid choice. Please try again.")
```

8.6 Flask Dashboard Routes

```
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/metrics")
def get_metrics():
    category = request.args.get("category", "All")
    city = request.args.get("city", "All")

    filtered = analyzer.filter_records(
        category=category, city=city
    )
    kpis = analyzer.get_kpis(filtered)
    monthly = analyzer.get_monthly_sales(filtered)
    categories = analyzer.get_category_sales(filtered)

    return jsonify({
        "kpis": kpis,
        "monthly": monthly,
        "categories": categories
    })
```

The full Flask application also supports category configuration updates and reset, simulated live transactions, text report download, filtered CSV export and CSV file upload. These features are implemented in app.py and are not all repeated in this short code extract.

The complete source code is included in the project ZIP: main.py, sales_analyzer.py, dashboard.py, generate_data.py and app.py, together with the HTML template, CSV data and configuration file.

9. OUTPUT AND DASHBOARD

The dashboard image below is the actual Matplotlib output generated from the uploaded project dataset and current category configuration. It contains four KPI cards and six visualizations.



9.1 Sample Executive Results

Metric	Result from supplied dataset
Total revenue	$142,205.66
Net profit	$60,270.64
Profit margin	42.4%
Transactions	1,201
Units sold	2,087
Average order value	$118.41
Unique customers	340
Average customer rating	4.64 / 5.00
Return rate	12.7%
Top category / product	Electronics / Smartphone

The supplied CSV contains 1,201 transaction rows. The totals above are recalculated from the CSV using the current category configuration. The project can produce different values if the dataset is regenerated, uploaded or category settings are changed.

10. CONCLUSION

The Retail Sales Dashboard Using CSV and Matplotlib project successfully demonstrates how Python can be used to process retail transaction data and convert it into meaningful business information. The system reads data from a CSV file, prepares the records and calculates important measures such as revenue, net profit, profit margin, average order value, return rate and customer rating.

The project analyses sales from different perspectives, including month, product category, city, payment method, product, customer segment and day of the week. Matplotlib is used to present these results through charts in a consolidated dashboard. The use of a GridSpec layout helps organize the KPI cards and multiple visualizations in a single figure.

The project also demonstrates modular programming through separate files for data generation, analysis, dashboard creation and application control. The command-line interface provides menu-based access to the functions, while the Flask web application adds browser-based interaction, filtering, CSV upload and export, report download, category configuration and live sale simulation.

The supplied dataset contains 1,201 transactions, with calculated revenue of $142,205.66 and net profit of $60,270.64 under the current configuration. These results illustrate how the application can summarize a large set of transaction records and make comparisons easier through visual representation.

Overall, this mini project combines CSV file handling, Python data structures, functions, classes, conditional statements, loops, exception handling, Matplotlib and Flask in a practical application. In the future, the project can be extended with a database, user authentication, real point-of-sale data, scheduled reports and more advanced forecasting methods.

PROJECT DETAILS

Project Topic: Retail Sales Dashboard Using CSV and Matplotlib
Student: HAIKRIVA R B
USN: 25BTRPW043
Class / Section: CSE-PWC(A)
Subject: Python Programming
Faculty: Ms. Sujithra G
Submission Date: 02/11/2026

— END OF REPORT —